

SP-4 Phase F.2 — Per-clone URL routing (implementation plan)

Plan date: 2026-05-13 **Design:** docs/superpowers/specs/2026-05-13-sp4-phase-f2-design.md (Option B locked: per-clone MirrorManager + SDK-side URL injection). **Prerequisite:** Phase F.1 complete (HEAD b54e7e63 and later).

Scope warning. This is a cross-module refactor across 6 tracker plugins. Every plugin's feature implementation (Search.kt, Browse.kt, Topic.kt, Auth.kt, Download.kt, Comments.kt, Favorites.kt) gains a URL-source parameter. Every plugin's existing per-feature unit tests must continue to pass. Recommended execution mode: focused dedicated session, NOT tail-end of an already-long multi-phase day. Validate each plugin's existing tests after touching it; rollback any plugin where the injection breaks coverage before proceeding to the next.

Goal (verbatim from F.2 design)

Make a cloned provider's `primaryUrl` (stored in `ClonedProviderEntity.primaryUrl`) actually route the clone's HTTP traffic, instead of routing through the source provider's URLs. Acceptance criterion: the Phase B clone-success Toast drops the "(URL routing pending — searches use source URLs)" disclosure once a clone of RuTracker at `https://rutracker.eu` produces a `MockWebServer` capture that contains `rutracker.eu` in the request URL.

Affected modules

- `core/tracker/rutracker` — primary refactor surface (most complex feature set).
- `core/tracker/rutor` — second-most complex; mirrors RuTracker's HTML-parsing shape.
- `core/tracker/nmclub` — simpler scrape; fewer feature impls.
- `core/tracker/kinozal` — similar to `nmclub`.

- core/tracker/archiveorg — apiSupported = true ; routes through Internet Archive JSON.
- core/tracker/gutenberg — also JSON-based; smallest surface.

Tasks

Task 1 — Define the URL injection seam

Files (new):

- core/tracker/api/src/main/kotlin/lava/tracker/api/MirrorUrlProvider.kt

package lava.tracker.api

```
/**
 * SP-4 Phase F.2 – per-call source of the base URL a tracker
 *   feature
 * implementation should hit. Injected by
 *   [TrackerClientFactory.create]
 * for clones (returning the clone's `primaryUrl` from
 *   `ClonedProviderEntity.primaryUrl`) and by Hilt's per-source-
 *   plugin
 * binding for original providers (returning
 *   `descriptor.baseUrls[0].url`).
 *
 * The provider is a function, not a constant, so future per-call
 * mirror-fallback can swap it without changing feature-impl
 * signatures.
 */
fun interface MirrorUrlProvider {
    fun baseUrl(): String
}
```

Steps:

☐

Create the file with the fun interface above.

☐

Add a compile-only unit test in :core:tracker:api's test source set asserting the interface can be implemented + invoked.

☐

Compile.

Task 2 — PluginConfig extension for clone URL override

Files:

- Modify: core/tracker/registry/src/main/kotlin/lava/tracker/registry/TrackerPluginConfig.kt (or wherever the Lava-domain config wrapper lives — confirm path first via `grep -rn "PluginConfig" core/tracker/registry/src/main`)

```
/** Convenience accessor: nullable clone URL override stamped by  
    LavaTrackerSdk.clientFor. */
```

```
val PluginConfig.cloneBaseUrlOverride: String?  
    get() = raw["lava.cloneBaseUrl"] as? String
```

Steps:



Add the extension property.



Compile.

Task 3 — LavaTrackerSdk.clientFor stamps the override

Files:

- Modify: core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt

```
private fun clientFor(id: String): TrackerClient {  
    val dao = clonedProviderDao  
    if (dao != null) {  
        val cloned = runBlocking { dao.getAll() }.firstOrNull {  
            it.syntheticId == id }  
        if (cloned != null) {  
            val sourceDescriptor = registry.list().firstOrNull {  
                it.trackerId == cloned.sourceTrackerId }  
            ?: throw IllegalStateException("Clone $id  
references unknown source ${cloned.sourceTrackerId}")  
            val cloneDescriptor = ClonedTrackerDescriptor(source  
= sourceDescriptor, override = cloned)  
            // NEW: pass the clone's primaryUrl to the factory via  
            PluginConfig.  
            val config =  
                MapPluginConfig(mapOf("lava.cloneBaseUrl" to  
                cloned.primaryUrl))  
            val sourceClient =  
                registry.get(cloned.sourceTrackerId, config)  
            return ClonedRoutingTrackerClient(sourceClient,  
                cloneDescriptor)  
        }  
    }  
}
```

```
        return registry.get(id, MapPluginConfig())
    }
```

Steps:



Modify `clientFor` to stamp `lava.cloneBaseUrl` into `PluginConfig` for clones.



Compile (the factories don't read it yet — they'll keep working with the source URL until each one is migrated in Task 4).

Task 4 — Per-tracker-plugin refactor

For each of the 6 plugins, in order (smallest scope first to validate the approach):

1. **gutenberg** (smallest — JSON-based, fewest features)
2. **archiveorg** (also JSON-based)
3. **kinozal**
4. **nnmclub**
5. **rutor**
6. **rutracker** (most complex — largest existing test surface)

Per-plugin sub-steps:



4.X.1 Read `<Plugin>Client.kt` + `<Plugin>ClientFactory.kt`. Identify where the source's base URL is captured today (typically `RuTrackerDescriptor.baseUrls[0].url` baked into an HTTP client at injection time).



4.X.2 Modify the factory: read `config.cloneBaseUrlOverride`; if non-null, construct a `MirrorUrlProvider` { `config.cloneBaseUrlOverride!!` }. Else `MirrorUrlProvider` { `descriptor.baseUrls[0].url` }.



4.X.3 Modify the client + its feature impls to accept the `MirrorUrlProvider` at construction time. Replace hardcoded `descriptor.baseUrls[0]` references in each feature's HTTP call with `urlProvider.baseUrl()`.



4.X.4 Run the plugin's existing unit tests (`./gradlew :core:tracker:<plugin>:testDebugUnitTest`). **MUST** all pass

before moving to the next plugin. If any break, ROLLBACK this plugin's changes and investigate before proceeding.



4.X.5 Add a falsifiability-rehearsal test per plugin: assert that when the factory is invoked with `MapPluginConfig(mapOf("lava.cloneBaseUrl" to "https://override.test"))`, a search HTTP call uses `https://override.test` (MockWebServer capture). Bluff-Audit stamp.

Task 5 — End-to-end Challenge Test

Files (new):

- `core/tracker/client/src/test/kotlin/lava/tracker/client/LavaTrackerSdkCloneRoutingTest.kt`

Drive the full path:

1. Seed a clone (`rutracker.clone.eu` → `https://rutracker.eu`).
2. Seed a MockWebServer at the override URL.
3. Invoke `sdk.multiSearch(SearchRequest("x"), listOf(clone.syntheticId))`.
4. Assert the `MockWebServer.takeRequest().requestUrl.host` is `rutracker.eu`, NOT `rutracker.org`.

Falsifiability rehearsal: revert Task 3's PluginConfig stamping; test fails because the source URL is hit instead.

Task 6 — Toast disclosure drop

Files:

- Modify: `feature/provider_config/src/main/kotlin/lava/provider/config/ProviderConfigViewModel.kt`

```
// BEFORE (Phase F.1, current state):
ProviderConfigSideEffect.ShowToast(
    "Cloned (URL routing pending – searches use source URLs)",
)
```

```
// AFTER (Phase F.2 acceptance):
ProviderConfigSideEffect.ShowToast("Cloned")
```

Steps:



Modify the Toast copy.



Compile.

Task 7 — CONTINUATION + commit + push

Steps:



Update docs/CONTINUATION.md §0 with the Phase F.2 delivery.



Commit with Bluff-Audit stamps for every per-plugin test added in Task 4.



Push to github + gitlab; verify SHA convergence.

Risk register

Risk	Mitigation
Plugin refactor breaks existing per-feature unit tests	Per-plugin validation gate before proceeding to next. ROLLBACK + investigate, not push-through.
<code>descriptor.baseUrls[0].url</code> is referenced elsewhere (e.g. probe URL, mirror health)	Grep before refactor: <code>grep -rn "descriptor.baseUrls\[0\]" core/tracker/</code> . Each callsite needs the same <code>urlProvider</code> injection OR explicit "this stays on the descriptor's base, not the clone's" reasoning.
MockWebServer test setup heavy per plugin	One shared test helper in <code>:core:tracker:testing</code> (already exists for fixture loading) gains a <code>mockWebServerFor(client)</code> builder.
Hilt scoping subtleties — singleton clients vs per-call clones	<code>RuTrackerClient</code> is <code>@Singleton</code> today (<code>@Inject</code> constructor + provided by Hilt). For clones, the factory must produce a NEW instance per call (or per clone synthetic id). Investigate scoping carefully — may need <code>@Provides</code> instead of <code>@Singleton</code> for the URL-injected variant.
<code>LavaMirrorManagerHolder</code> already exists for the source's mirror-fallback	Per-clone mirror manager construction needs to compose with this. Defer to a Phase F.3 follow-up if

Risk	Mitigation
	the integration is non-trivial; F.2 ships single-URL routing first.

Acceptance gates

Phase F.2 is COMPLETE when ALL of:

1. Every per-plugin test gate (Task 4) passes.
2. The end-to-end Challenge Test (Task 5) passes.
3. The Toast copy (Task 6) is "Cloned" (no parenthetical disclosure).
4. The CHANGELOG entry documents Phase F.2 as the user-visible "Clones now actually use their custom URL" deliverable.
5. The mirror SHAs converge on both github + gitlab post-push.

Estimated session time

- Task 1 + 2 + 3: ~30 minutes (small + low-risk).
- Task 4: **6 × 45-90 minutes per plugin = 4.5–9 hours**. This dominates the schedule.
- Task 5 + 6 + 7: ~45 minutes combined.

Total realistic estimate: 6–11 hours of focused work. A single-session execution is feasible only on a dedicated day with no other work in flight; otherwise stage across 2–3 sessions, one plugin pair per session, with a passing-tests gate between sessions.